ALBart: Navigating Music Through Embedding Space

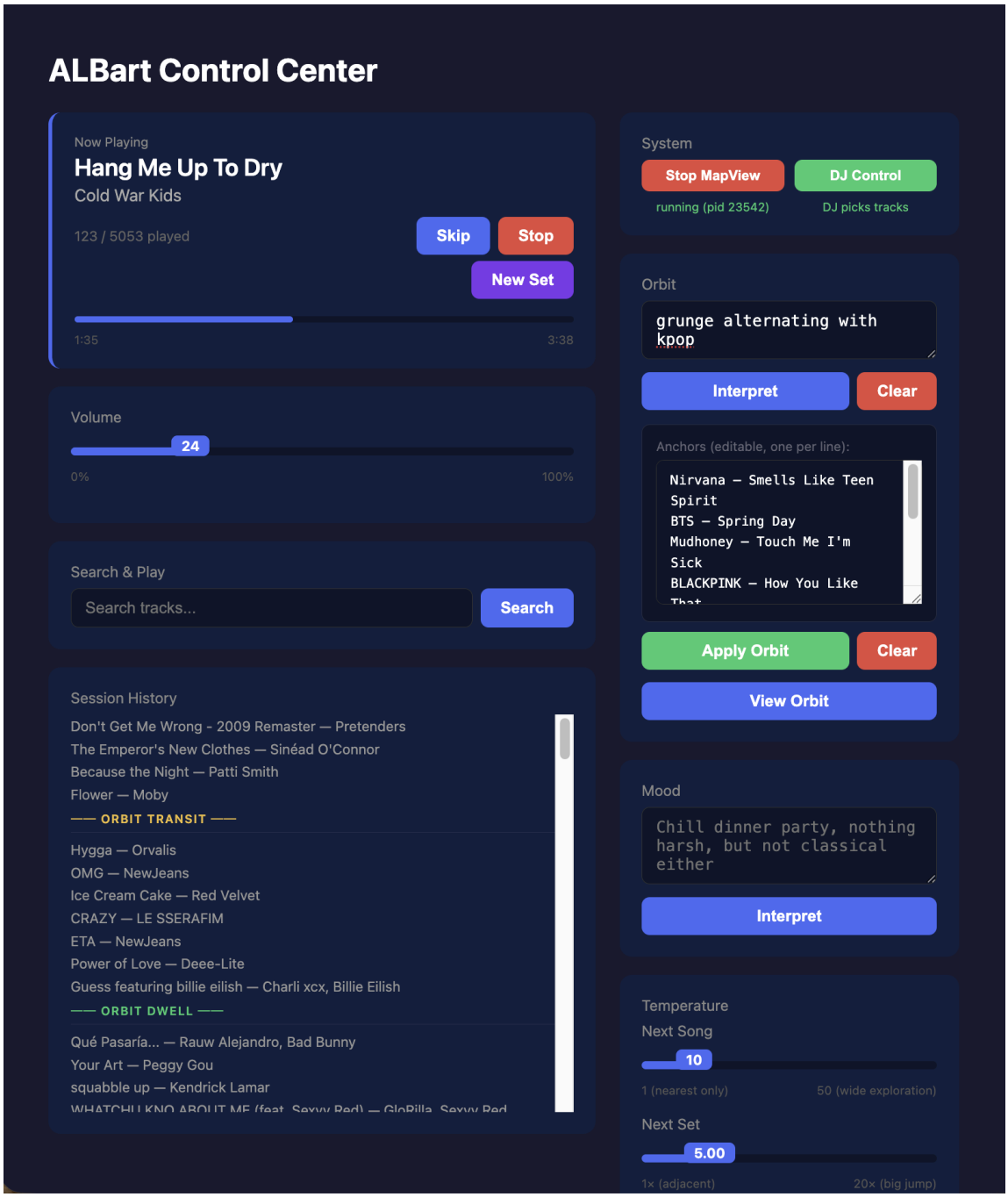
An AI DJ System for Exploring Audio Similarity

1. Introduction

ALBart is an AI DJ that treats a music library not as a list of songs sorted by genre, artist, or mood, but as a cloud of points in high-dimensional space. Each track is represented by a 512-dimensional vector that captures its sonic character — timbre, energy, production style, rhythmic texture. Tracks that sound similar sit close together. Tracks that sound different are far apart.

This simple idea — music as geometry — unlocks a new way of exploring a collection. Instead of browsing by genre or searching by name, the DJ navigates through the space, hopping from one track to its nearest neighbors, drifting through regions of similarity, occasionally making long jumps to unexplored territory. The result is a continuous musical journey that respects sonic affinity while surfacing connections that genre labels would never reveal: Howlin' Wolf next to Jonathan Richman, Ray Charles adjacent to The Meters, David Bowie's "Young Americans" surrounded by Stevie Wonder and Funkadelic rather than by "Heroes" or "Space Oddity."

The system includes a real-time 3D visualization (MapView) that shows the DJ's current position in embedding space, surrounded by album art thumbnails of nearby tracks. A web-based control center provides mood filtering, orbit navigation (guided journeys through the space), and playback control. The entire system is designed around a clean separation of concerns that makes it portable to different music sources and players.



Screenshot: ALBart Control Center

2. How Tracks Become Vectors

The foundation of the system is CLAP (Contrastive Language-Audio Pretraining), an open-source model from LAION that maps audio into a 512-dimensional embedding space. CLAP was trained on hundreds of thousands of audio-text pairs, learning to place audio near text descriptions that match it. As a side effect, it learned a rich representation of audio similarity: tracks with similar

sonic characteristics end up near each other in the 512D space, even if they come from different genres, eras, or cultures.

To embed a track, ALBart splits the audio into non-overlapping 10-second chunks, runs each through CLAP, and averages the resulting vectors. A 30-second preview yields 3 chunks; a full 4-minute track yields 24, giving a more representative embedding that captures intro, verse, chorus, bridge, and outro. The averaged vector is L2-normalized to a unit sphere, making distance comparisons meaningful.

Before CLAP inference, the audio undergoes dynamic range compression and a 4kHz low-pass filter. This preprocessing was empirically tuned to improve consistency across different recording qualities — a lo-fi bedroom recording and a polished studio master of the same song produce more similar embeddings with this normalization than without it.

Embedding is fast: approximately 0.1 seconds per track on Apple Silicon (MPS), including file loading, resampling, preprocessing, and inference. A 5,000-track library embeds in about 8 minutes; a million tracks would take roughly 27 hours on a single machine.

3. Taming 512 Dimensions

The 512D CLAP space captures audio texture with high fidelity, but it has a problem for navigation: it groups tracks by how they sound at a production level, not by what genre or style they belong to. A lo-fi indie track and a lo-fi hip-hop track might be closer in 512D than two indie tracks with different production styles. When the DJ dwells near an anchor track in 512D, it tends to drift across genres, following production similarity rather than musical genre.

UMAP Projection: 512D → 25D

UMAP (Uniform Manifold Approximation and Projection) is a dimensionality reduction technique that preserves both local and global structure. By projecting the 512D embeddings down to 25 dimensions with carefully chosen parameters (n_neighbors=75, min_dist=0.3, cosine metric), the resulting space captures genre-level structure while discarding the production-level noise that makes 512D navigation unpredictable.

The difference is dramatic. In 25D, dwelling near a Ray Charles anchor stays in soul and R&B. In 512D, the same dwell drifts into ambient electronic because some Ray Charles recordings share production characteristics with ambient music. The 25D space preserves the meaningful relationships and filters out the coincidental ones.

Parametric UMAP for Real-Time Projection

Standard UMAP is a batch algorithm — it needs the full dataset to compute projections. This is fine for the initial library, but ALBart needs to project new tracks in real time (e.g., when a user plays an unknown track on Spotify). The solution is a parametric UMAP: a small neural network (3-layer MLP, 512→256→256→25) trained to reproduce the UMAP output. Once trained, it can project any new 512D embedding to 25D in microseconds.

The MLP is trained by distillation: run standard UMAP on the full library, then train the MLP to regress from 512D inputs to the UMAP 25D outputs. On a held-out validation set, the mean L2 error is 2.7% of the coordinate diagonal — the projected points land very close to where full UMAP would place them. After large library additions, the UMAP and MLP can be retrained in about a minute for 5,000 tracks.

4. Navigating the Space

The DJ engine navigates through the 25D space using several strategies, all implemented as pure functions that take immutable state and return new state plus commands. This functional architecture makes the navigation logic fully testable without any infrastructure.

Normal Hops

The default navigation: find the K nearest unplayed tracks in 25D, weight them by inverse-cube distance (strongly favoring the nearest), apply an artist penalty to avoid back-to-back plays by the same artist, and pick one at random. The cube weighting means the nearest track gets about 32% probability, the top 3 get about 66%, but the full pool of 10 candidates allows for occasional surprises.

Long Hops (Set Changes)

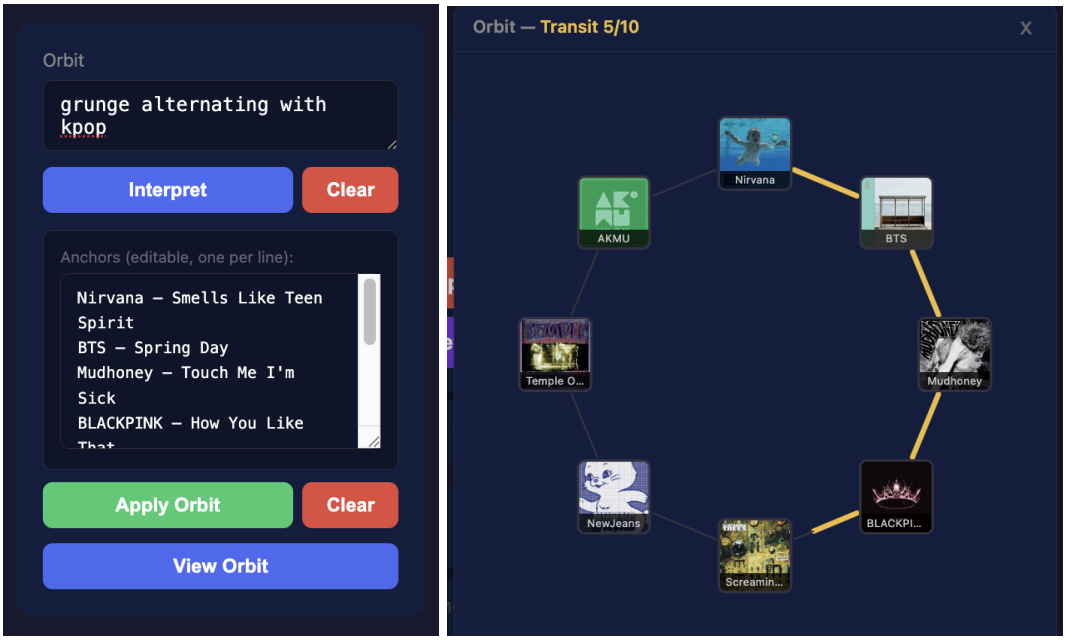
Periodically (default: every 30 minutes), the DJ extrapolates its recent trajectory in 25D space and jumps to a distant point, landing in a new genre region. This prevents the DJ from getting stuck in one corner of the space and creates natural "set" boundaries — the musical equivalent of a DJ switching from jazz to electronic.

Orbit Navigation

The most distinctive feature. An orbit is a guided journey through the space, defined by anchor tracks chosen by an LLM (Claude) based on a natural-language description like “trace the evolution of David Bowie” or “alternate between grunge and K-pop.” Claude selects tracks from the library that define the journey’s waypoints and determines whether same-artist runs should be allowed — a Ray Charles deep dive permits consecutive Ray Charles tracks; a genre survey prefers variety. The orbit operates as a two-phase state machine:

Dwell phase: The DJ plays tracks near the current anchor, exploring its neighborhood for a configurable duration (default 30 minutes). Anchor tracks serve as attractors but are never played directly — the DJ plays tracks near them. This prevents iconic anchor tracks from being overplayed across repeated orbits, and ensures that each pass through the same anchor discovers different tracks.

Transit phase: The DJ moves from the current anchor toward the next one, interpolating through 25D space. Each step targets a fraction of the distance from the current track’s actual position to the destination anchor. Because the DJ picks from neighbors near each interpolation point (not along a fixed line), the path through the space is different every time — there isn’t just one route from grunge to K-pop.



Screenshot: Orbit viewer showing anchors with album art, transit progress bars, and hover tooltip

5. The MapView: Seeing the Space

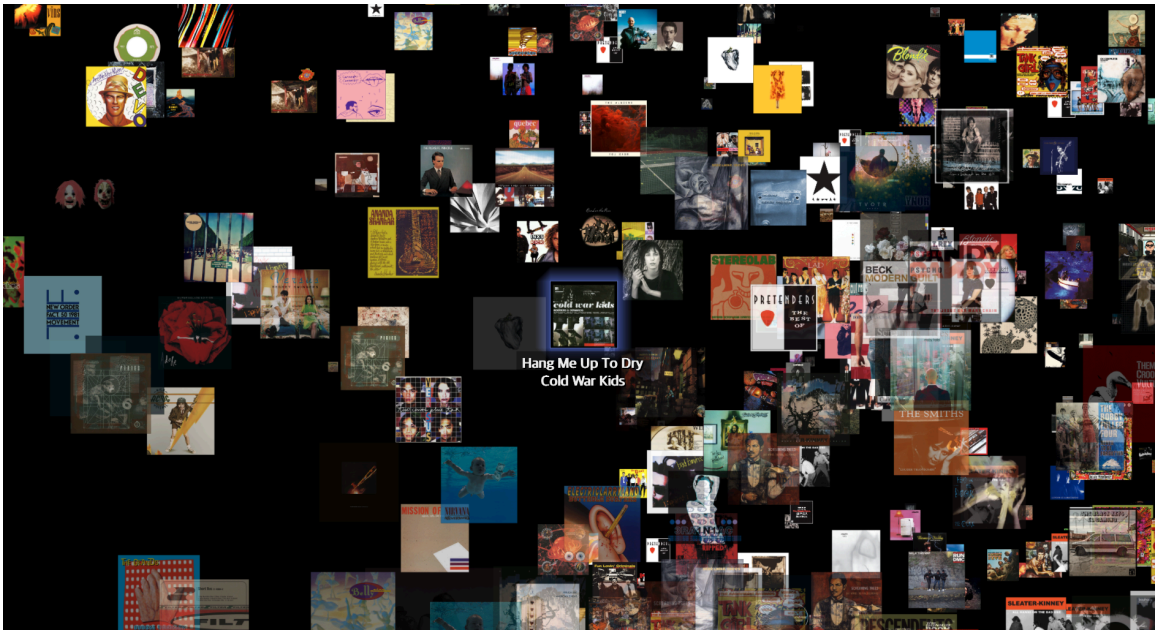
The MapView is a real-time 3D visualization of the DJ's current position in embedding space. It solves a fundamental challenge: how do you show a 25-dimensional space on a 2D screen?

Local PCA Projection

The approach is a hybrid: use the full 25D for distance (how far away each track is) and a local PCA for direction (which way). For each update, the system finds the K nearest neighbors in 25D, computes PCA on that local neighborhood to extract the 3 most informative dimensions, and projects all tracks into that 3D subspace. The result adapts to the current region — different parts of the space reveal different structure.

Album art thumbnails are rendered as OpenGL-textured quads at their projected positions. The current track sits at the center, marked by a glowing sphere. Nearby tracks appear larger (perspective projection with configurable focal length), and the camera rotates slowly to give a sense of the 3D structure. As the DJ moves through the space, the view smoothly lerps to the new position, and the PCA basis is periodically recomputed to keep the projection fresh.

The track title and artist are rendered above all album covers, ensuring readability. Hovering over any thumbnail shows a tooltip with the full-size art, title, and artist. The font (Apple SD Gothic Neo) supports Latin, Korean, Japanese, and Chinese characters.



Screenshot: MapView showing 3D neighborhood with album art, current track label, and nearby thumbnails

6. Mood and Genre Filtering

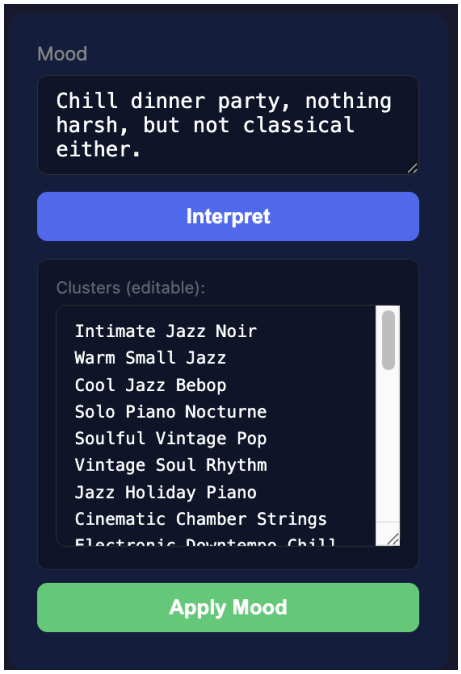
The DJ can be constrained to play only tracks matching a mood or genre description. Early experiments used CLAP's text-to-audio matching to compare mood descriptors against track embeddings, but the results were unreliable — CLAP's text understanding wasn't designed for abstract mood concepts like "chill dinner party."

Cluster-Based Filtering

The current approach uses k-means clustering in 25D space to discover natural genre groups, then asks Claude to label each cluster from sample tracks. For a library of 5,000 tracks, this produces about 39 clusters with labels like "Solo Piano Classical," "Punk Hard Rock," "Electronic Downtempo Chill," and "Soulful Vintage Pop."

The number of clusters scales as a power law of library size ($k \approx N^{0.43}$), reflecting the hierarchical nature of genre: adding more tracks fills in existing clusters and occasionally opens up new subgenres, but doesn't linearly increase the number of distinct categories.

When the user sets a mood (e.g., "chill dinner party, nothing harsh, but not classical either"), Claude selects which clusters to include and exclude from the actual cluster vocabulary. The filter then includes any track within the mean radius of an included cluster's centroid, and excludes tracks within the mean radius of any excluded cluster. Positive inclusion happens first, then negative exclusion — a track near both an included and excluded cluster is removed.



Screenshot: Mood filter UI showing cluster labels as include/exclude descriptors

7. Growing the Library

ALBart supports several ways to add tracks to the library, all converging on the same pipeline: audio → CLAP embedding → 25D UMAP projection → PostgreSQL with pgvector indexes.

Spotify Top Tracks

The original pipeline pulls a user's top tracks from Spotify's listening history (typically 5,000+ tracks for active listeners), downloads 30-second preview clips, and computes embeddings. This provides the initial library.

On-the-Fly Ingestion and Spotify Control

When the DJ encounters an unknown track — whether it picked the track itself or the user queued it from Spotify — the system automatically ingests it in a background thread: fetch metadata from the Spotify API, download the preview, compute the CLAP embedding, project to 25D, and store in PostgreSQL. This typically completes in 15–20 seconds, well before the track finishes playing. The MapView shows the track's title and artist immediately and snaps to the correct position once the embedding is ready. A "Spotify Control" mode lets the user drive playback manually while the system follows along, ingesting every unknown track it encounters and broadcasting positions to the MapView. This is a convenient way to build the library from existing playlists without waiting for the batch tool.

Batch Playlist Import

A command-line tool ingests entire playlists or albums, processing tracks in parallel (4 download workers by default, sequential CLAP inference). The tool accepts Spotify URLs, URIs, or text files with one track URL per line. It's idempotent — tracks already in the database are skipped.

8. Architecture and Portability

ALBart is designed around a clean separation of concerns that makes the system portable to different music sources and players.

Functional Core

All navigation logic lives in pure functions that take immutable state and return new state plus command objects. No I/O, no side effects, no time.monotonic() calls — time is a parameter. This layer has 78 unit tests that run in under 200ms with zero infrastructure. The commands (PlayTrackCommand, FindNeighborsCommand, BroadcastToMapCommand, etc.) are frozen Pydantic models — just data describing desired side effects.

Effects Layer

All side effects (Spotify API, database queries, UDP broadcast, CLAP inference) are isolated in effect modules. Effects never modify DJ state — they execute commands and return results. The database uses PostgreSQL with pgvector for vector search, with HNSW indexes that provide >99% recall at million-track scale.

PlaybackClient Protocol

The engine interacts with the music player through a minimal protocol: poll_playback, play_track, resume, pause, seek, set_volume. The Spotify implementation is one concrete class; porting to a different player (e.g., MPD for a local file library) requires implementing these six methods. The engine, pure logic, MapView, and web UI all work unchanged.

Similarly, track ingestion is separated from the playback backend. The CLAP embedding pipeline, UMAP projector, and database client are all player-agnostic. A port to a local file library would need a directory scanner and metadata reader (e.g., reading tags with mutagen), but the embedding and storage code is reusable as-is.

9. What's Next

Several directions are natural extensions of the current system:

Larger libraries. The current library of ~5,000 tracks occupies a small fraction of the embedding space. With tens or hundreds of thousands of tracks, the genre clusters would become finer-grained, the transit paths more varied, and the discovery of unexpected connections more frequent. The database and indexes are designed for million-track scale; the UMAP model would need periodic retraining as the library grows significantly.

Non-Spotify sources. The PlaybackClient abstraction makes it straightforward to port ALBart to a local file collection managed by MPD or similar. For large private collections (including lossless formats, rare recordings, or content not available on streaming services), the full-track embedding from high-quality audio would likely produce even better results than the 30-second Spotify previews the system was developed with.

Room audio feedback. An experimental RoomEar module captures ambient audio from a microphone, computes CLAP embeddings, and broadcasts them for visualization. The domain gap between studio recordings and room audio currently limits matching accuracy, but the approximate genre-level positioning works and could enable interesting interactive scenarios — the DJ responding to what the room sounds like rather than what Spotify says is playing.

Physical display. ALBart was originally designed for a 32×32 LED matrix displaying album art that matches ambient audio. The display backend is abstracted behind a simple interface (show_frame(rgb_array)), and a HUB75 LED panel driver exists for Raspberry Pi deployment. The combination of the LED display showing what the room hears and the MapView showing where the DJ is navigating creates a compelling ambient installation.